

Spring Boot: Builder, DTOs e Mappers

Autores: Andrey Mota de Oliveira
Gabriel da Silva e Silva
Giovanna Oliveira Araujo
João Magalhães Mira Neto
Louise de Souza Santino

Agenda

O objetivo desta apresentação é demonstrar a evolução da arquitetura desenvolvida para a Web API da MecâniQA, conectando os padrões de projeto estruturais e criacionais (Builder).

1. Contextualização:

A evolução do MVP da MecâniQA na Fase 2.

2. Conceitos Básicos de Orientação a Objetos:

Múltiplos construtores e o uso do *this*.

3. Relacionamentos e Multiplicidades :

Tipos de relacionamentos, multiplicidades aplicadas e como os atributos são usados.

4. Conceitos de Diagrama de Classes:

Setas, diamantes de todo-parte, conceito de dependência e como funciona uma entidade associativa.

5. Padrões de Projeto :

Builder, Mapper e DTO.

6. Considerações Finais e Referências:

Oportunidades de melhoria encontradas e material de apoio para embasamento.

Contextualização



- A **MecâniQA** é uma empresa de soluções, voltada ao mercado automotivo.
- Tivemos o backend inicial com catálogo de peças/serviços e armazenamento em memória.
- O foco agora é o desacoplamento da API através de DTOs/Mappers e instanciação segura de objetos complexos com o Padrão Builder.



Conceitos Básicos – Sobrecarga

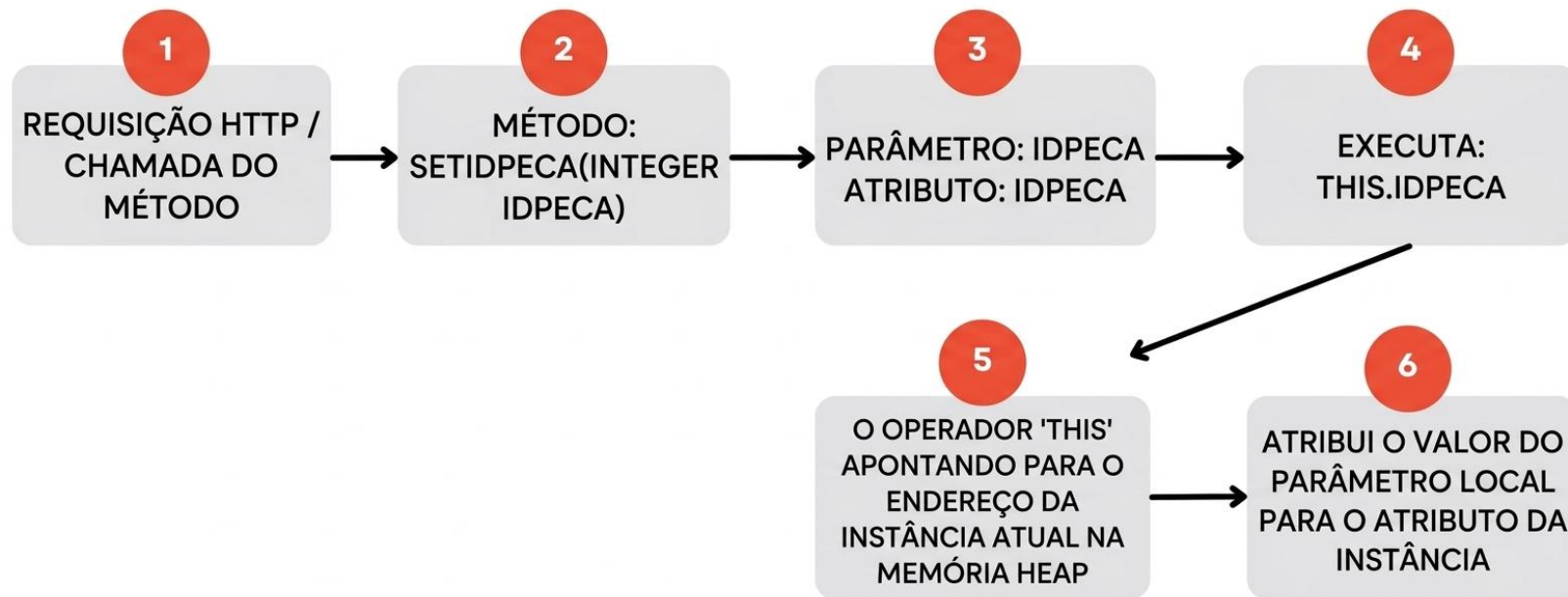


- Instancia objetos com diferentes conjuntos de dados iniciais.

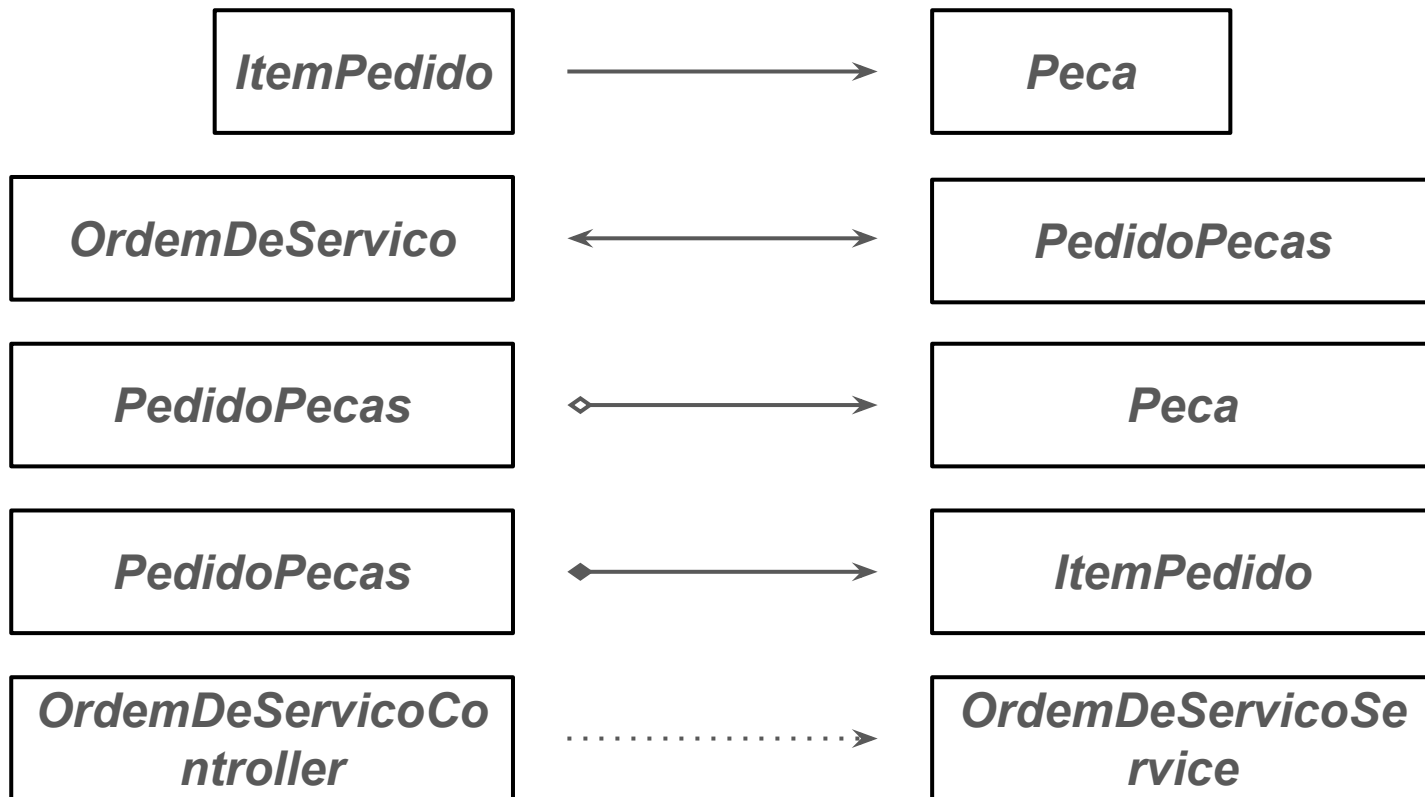


Ex: PecaDTO: responsável pela serialização/deserialização.

Conceitos Básicos - This



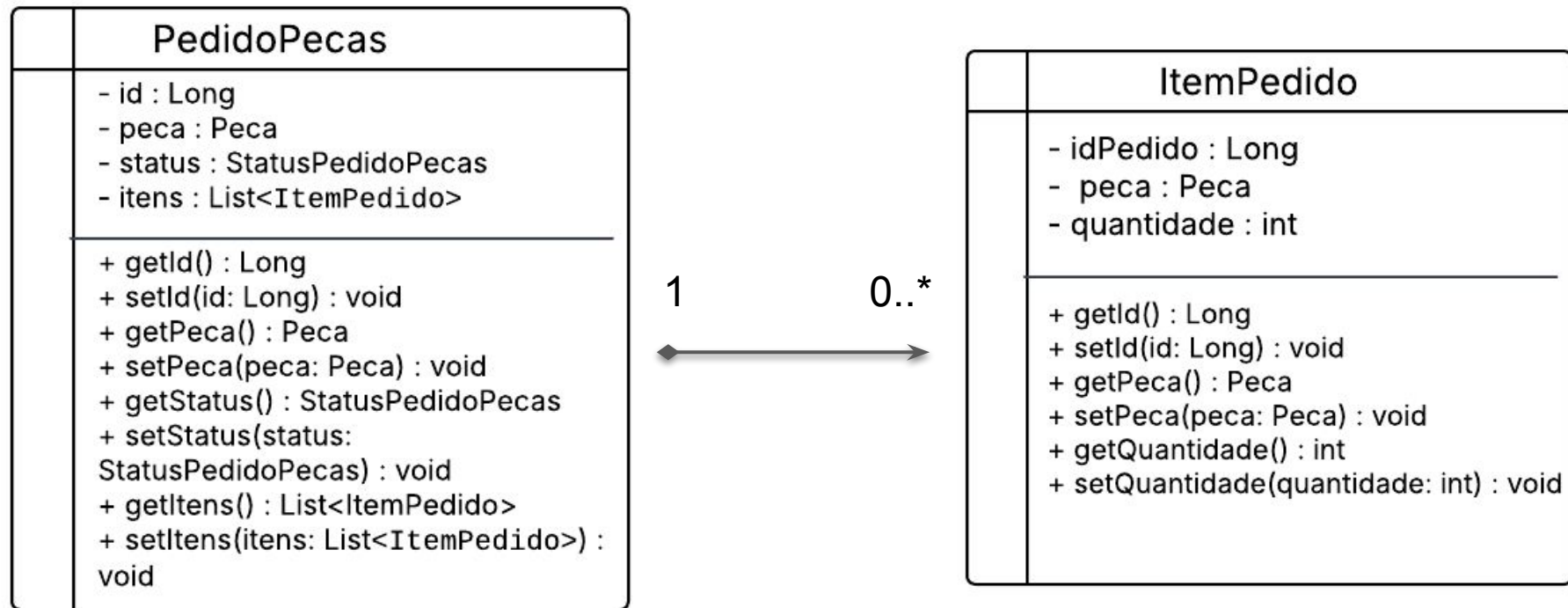
Relacionamentos



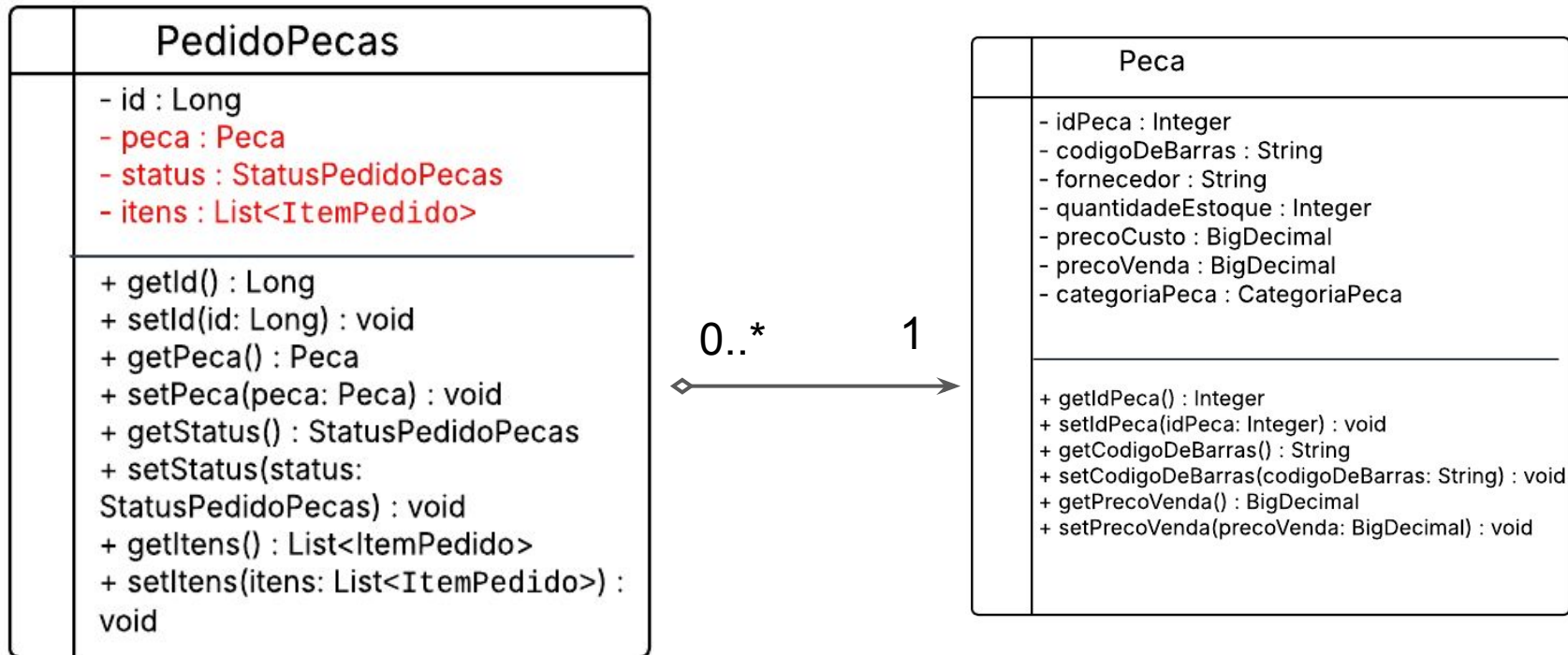
Multiplicidades

1 ou 1..1	Exatamente um
0..1	Zero ou um
1..*	Um ou muitos
0..* ou *	Zero ou muitos
n	Número fixo
n..m	Intervalo fixo

Relacionamentos - Composição



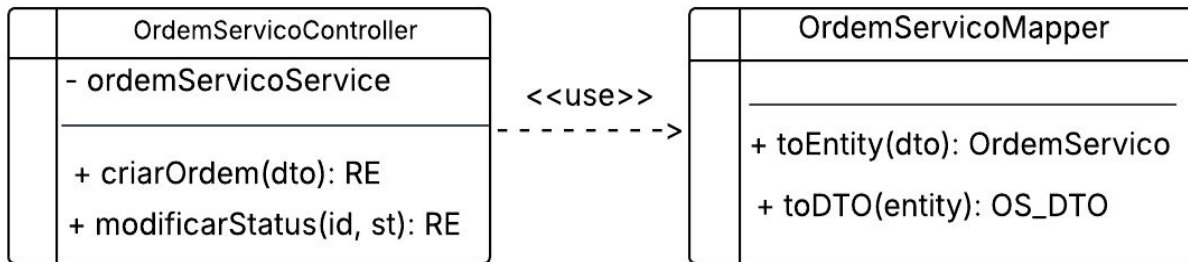
Relacionamentos - Agregação



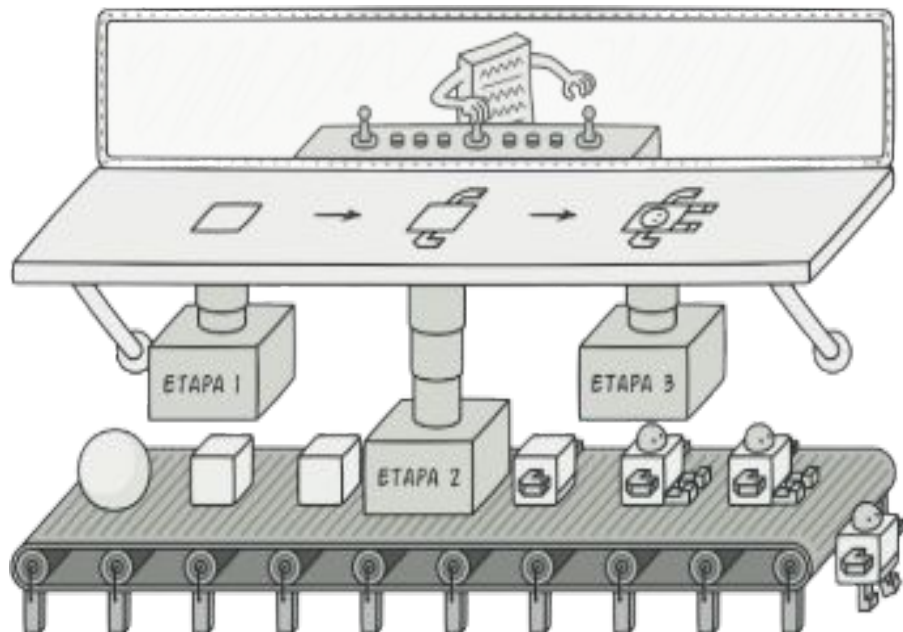
Relacionamentos – Dependência



- Relação de Uso Temporário.
- Ausência de Atributo de Instância.
- Dependência de compilação.

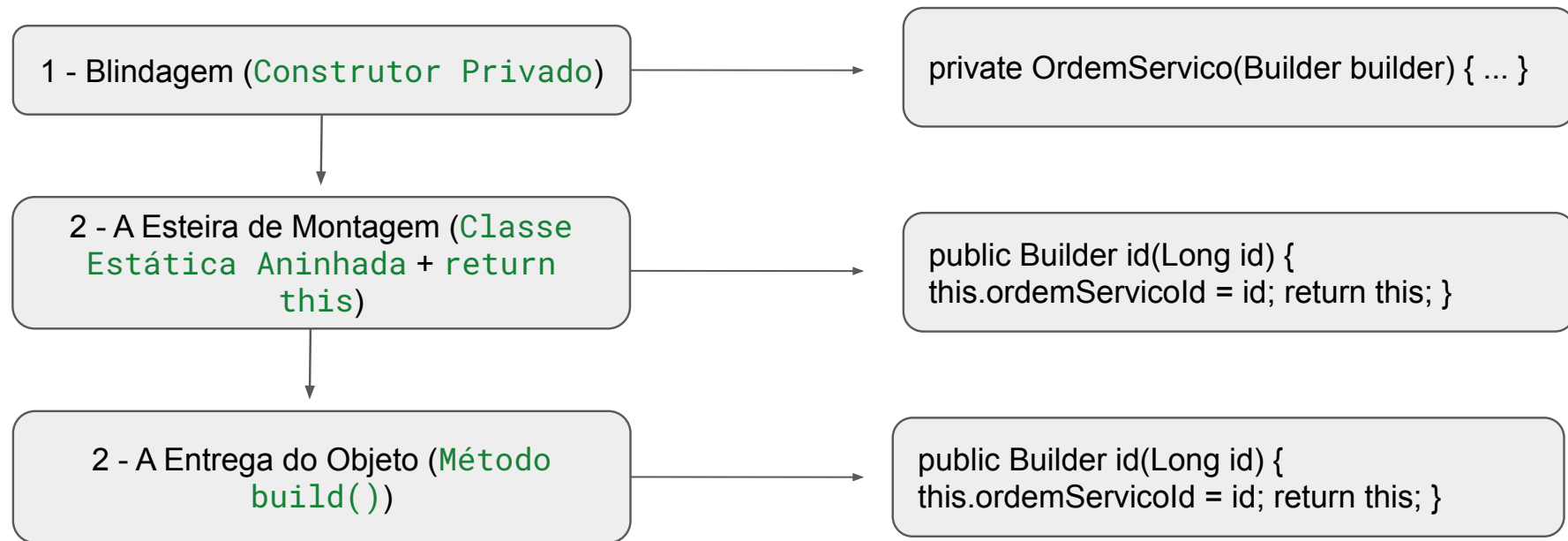


Builder



- Separa a criação de um objeto de sua estrutura.
- Evitar construtores poluídos e criar objetos de forma fluida.

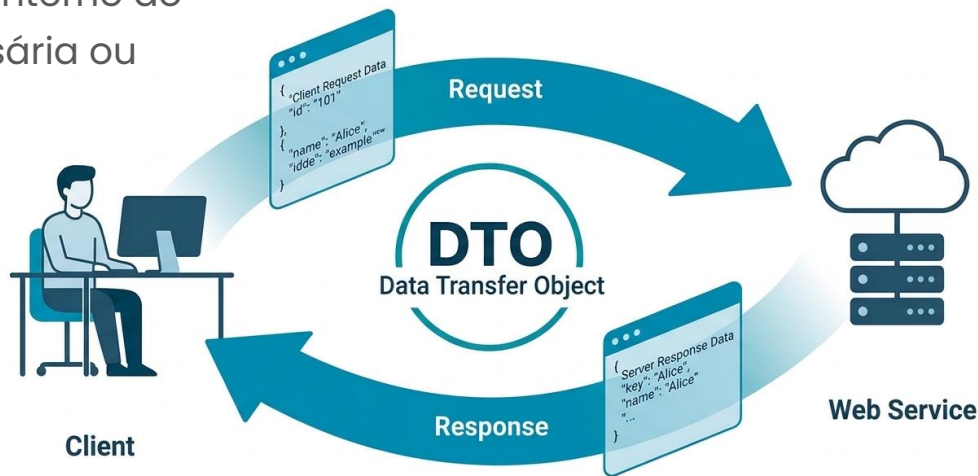
Builder – Mecânica



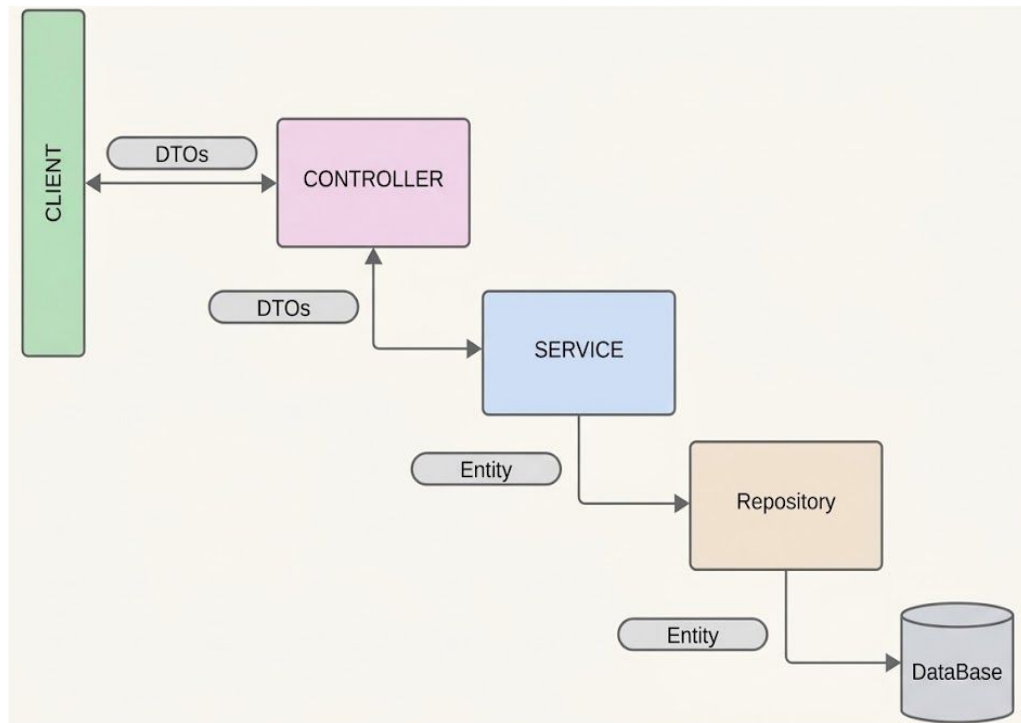
DTO

- Objeto simples que transporta dados entre processos ou camadas sem conter lógica de negócio.
- Tem como objetivo proteger o modelo interno do sistema e evitar a exposição desnecessária ou vazamento de dados sensíveis.

Ex: Evita a separação entre dados de entrada e dados de saída da API.



DTO

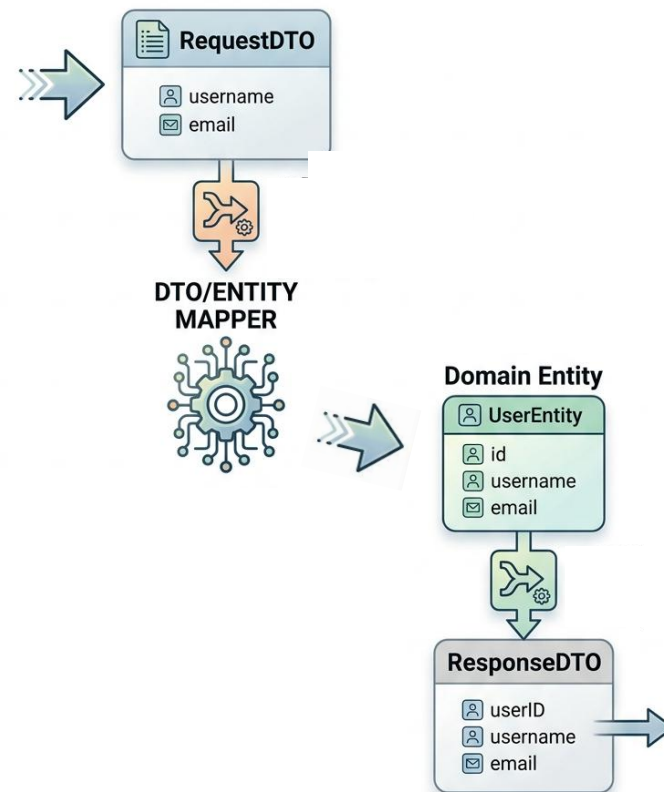


Mapper

- Classe utilitária com métodos estáticos responsável pela conversão de dados, ela isola a responsabilidade de transformação do *Controller* e do *Service*.

Ex: *toEntity(dto)* → Transforma *RequestDTO* em Entidade de domínio.

toDTO(entity) → Transforma Entidade em *ResponseDTO*.



Considerações finais



- A API da MecâniQA avançou de um modelo básico para uma estrutura robusta em Spring Boot.
- O uso de DTOs e Mappers isolou as entidades de domínio, enquanto o padrão Builder garantiu a criação segura das Ordens de Serviço sem o uso de Lombok.
- A inclusão da entidade associativa ItemPedido e o mapeamento correto dos relacionamentos (Agregação/Composição) refletem com fidelidade as regras do negócio.

Referências



- SIERRA, Kathy; BATES, Bert; GEE, Trisha. Use a cabeça! Java. 3. ed. Rio de Janeiro: Alta Books, 2023.
- DEITEL, Paul; DEITEL, Harvey. Java: como programar. 10. ed. São Paulo: Pearson Education do Brasil, 2016.
- ECKEL, Bruce. Pensando em Java. 3. ed. rev. 4.0. Rio de Janeiro: Alta Books, 2005.
- GAMMA, Erich et al. *Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos*. Porto Alegre: Bookman, 2000.
- OBJECT MANAGEMENT GROUP (OMG). *Unified Modeling Language (UML) Specification*. Versão 2.5.1, 2017.
- PRESSMAN, Roger S.; MAXIM, Bruce R. *Engenharia de Software: Uma Abordagem Profissional*. 8. ed. Porto Alegre: AMGH, 2016.
- SPRING FRAMEWORK. *Spring Boot Reference Documentation*. Pivotal Software, 2026. Disponível em: <https://spring.io/projects/spring-boot>.